

Cookie Auth

Key Concepts

- **Cookie Auth** — browser automatically attaches a session cookie to every request; no client code needed
- **Session ID** — random identifier stored in the cookie; actual session data lives server-side
- **Stateful** — server holds session data (userId, roles, cart, etc.) keyed by session ID
- **CSRF** — biggest security risk; browser auto-sends cookies so attackers can exploit it cross-origin
- **HttpOnly flag** — prevents JavaScript from reading the cookie (blocks XSS attacks)
- **Secure flag** — cookie only sent over HTTPS

How It Works

1. User logs in
2. Server creates a session, stores session data in DB/memory: `sessionId → { userId, role, ... }`
3. Server returns: `Set-Cookie: sessionId=abc123; HttpOnly; Secure`
4. Browser stores cookie automatically
5. Every subsequent request: browser auto-attaches `Cookie: sessionId=abc123`
6. Server looks up `sessionId` → retrieves session data → authenticates user

```
Login:
Client --> POST /login (user+pass) --> Server
                                create session in DB
                                sessionId=abc123 → { userId: 42 }
Client <-- Set-Cookie: sessionId=abc123 <-- Server

Subsequent requests:
Client --> GET /dashboard
        Cookie: sessionId=abc123 --> Server
                                lookup sessionId in DB
                                retrieve session data

Client <-- 200 OK <-- Server
```

Security Flags

Flag	Purpose
HttpOnly	JS cannot access cookie — blocks XSS token theft
Secure	Cookie only sent over HTTPS
SameSite=Strict	Cookie not sent on cross-origin requests — blocks CSRF
SameSite=Lax	Cookie sent on top-level navigation only

CSRF — The Key Risk

- Browser auto-sends cookies on **any** request to the domain — even from another site
- Attacker on `evil.com` can trick your browser into hitting `bank.com/transfer` with your valid cookie
- **Fix: CSRF Token** — server issues a random value that must be included in state-changing requests (POST, DELETE); attacker can't forge it cross-origin

Cookie Auth vs JWT

	Cookie Auth	JWT
Storage	Server (session store)	Client (token)
State	Stateful	Stateless
Scalability	Harder (shared session store needed)	Easy
Revocation	Easy (delete session)	Hard
Best for	Traditional web apps, browsers	APIs, mobile, microservices

Industry Examples

- **Amazon** — session cookies maintain cart and login state across browser sessions
- **Banking apps** — session-based for server-side control and instant revocation
- **Traditional MVC apps** (Rails, Django, ASP.NET MVC) — cookie sessions by default

Interview Questions

Q: How does Cookie Auth work?

On login, server creates a session and stores data server-side. It returns a Set-Cookie header with a session ID. The browser automatically sends that cookie on every subsequent request. The server looks up the session ID to identify the user.

Q: What is CSRF and how do you prevent it?

CSRF exploits the fact that browsers auto-send cookies — an attacker on another site can trigger requests to your app using the victim's valid session. Prevention: CSRF tokens (server issues a random value that must accompany state-changing requests; can't be forged cross-origin) and `SameSite` cookie attribute.

Q: When would you choose Cookie Auth over JWT?

For traditional server-rendered web apps where you need easy revocation (logout, account ban), session-level data storage, and browser-native behavior. JWT is better for stateless APIs, mobile clients, and microservices where horizontal scaling is a priority.

Q: What does HttpOnly do?

Prevents JavaScript from reading the cookie via `document.cookie`. This blocks XSS attacks from stealing the session token even if an attacker injects malicious JS into the page.